Comprehensive Notes on Image Processing Concepts

Computational Vision Study Guide

1 Linear Filtering in Space

1.1 Informal Explanation

Think of linear filtering as sliding a small grid of numbers (called a "kernel" or "mask") over an image to calculate a new value for every pixel. Depending on the numbers in the kernel, this process can average out noisy pixels, sharpen blurred details, or enhance certain features like edges.

1.2 Formal Mathematical Formulation

In linear cases, filtering corresponds to applying a mathematical operation called convolution to an image signal f. If f is the original image and k is the kernel, the filtered image g is defined continuously as $g = f * k$.

For a discrete, two-dimensional digital image, convolution in the space domain is formulated as:

$$g[x, y] = \sum_m \sum_l f[x - m, y - l]k[m, l]$$

Breaking down the terms:

- $g[x, y]$: The new, filtered pixel value at the specific row x and column y in the output image.
- $f[x-m, y-l]$: The pixel value in the original image. The subtraction of m and l represents shifting the focus around the target pixel to look at its neighbors.
- $k[m, l]$: The weight assigned by the kernel at position (m, l). The kernel acts as a filter matrix; the size of this matrix dictates the bounds of the summation (e.g., a 3×3 kernel means m and l range from -1 to 1).
- **The mechanism:** For every pixel (x, y), the equation multiplies the neighboring pixels by the corresponding weights in the kernel, sums them all up, and assigns that single total value to the new pixel $g[x, y]$.

2 Image Gradient

2.1 Informal Explanation

If you imagine an image as a 3D topographic map where bright pixels are high mountains and dark pixels are deep valleys, the image gradient tells you two things about any spot on the map: the direction of the steepest uphill slope, and exactly how steep that slope is.

2.2 Formal Mathematical Formulation

The gradient is a vector composed of partial derivatives. It points in the direction of the most rapid change in image intensity:

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Breaking down the terms:

- ∇f: The gradient vector of the image f.

- $g_x = \frac{\partial f}{\partial x}$: The partial derivative with respect to x. It measures how rapidly the pixel intensity changes as you move horizontally. A high value means a strong vertical edge.
- $g_y = \frac{\partial f}{\partial y}$: The partial derivative with respect to y. It measures how rapidly the pixel intensity changes as you move vertically. A high value means a strong horizontal edge.

From this vector, we derive two critical measurements:

1. **Gradient Magnitude (Steepness):**

$$M(x, y) = \sqrt{g_x^2 + g_y^2}$$

This uses the Pythagorean theorem to calculate the total length of the gradient vector. A large magnitude indicates a sharp, distinct edge.

2. **Gradient Orientation (Direction):**

$$\theta(x, y) = \arctan\left(\frac{g_y}{g_x}\right)$$

This calculates the angle of the edge. It tells us the direction of the steepest change in intensity relative to the horizontal axis.

3 Filters and the Effect of Noise

3.1 Informal Explanation

When trying to find changes in an image (like edges), taking a mathematical derivative is dangerous because derivatives measure *any* change, including random static (noise). To fix this, we first blur or "smooth" the image to average out the static, making the true, structural changes easier to detect.

3.2 Formal Mathematical Formulation

We smooth the signal using a Gaussian filter, which acts as a low-pass filter to remove high-frequency noise. The 2D Gaussian function is:

$$h_\sigma(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

Breaking down the terms:

- $h_\sigma(u, v)$: The weight calculated for a pixel located at distance (u, v) from the center of the kernel.
- $e^{-\frac{u^2+v^2}{2\sigma^2}}$: The core exponential decay function. Because $u^2 + v^2$ is the squared distance from the center, this term ensures that pixels further from the center receive exponentially less weight.
- σ (Sigma): The standard deviation. It controls the "spread" or "fatness" of the Gaussian bell curve. A larger σ means heavier blurring because farther pixels retain more weight.
- $\frac{1}{2\pi\sigma^2}$: A normalization constant. It ensures that the sum of all weights in the continuous Gaussian distribution equals exactly 1, preventing the image from becoming artificially brighter or darker after filtering.

By the associative property of convolution, we can combine the smoothing and the derivative step into a single operation, saving computational time:

$$\frac{\partial}{\partial x}(h * f) = \left(\frac{\partial}{\partial x} h\right) * f$$

4 Enhancement Filters (Derivative of Gaussian)

4.1 Informal Explanation

Instead of blurring the image and *then* looking for edges, we can take the mathematical derivative of our blur filter to create a single "Enhancement Filter." This combined tool ignores static while actively highlighting vertical, horizontal, or angled lines in one pass.

4.2 Formal Mathematical Formulation

We derive basis filters by taking the derivative of the Gaussian formula.

The first derivative of a Gaussian with respect to x (sensitive to vertical structures) is:

$$G_1^{0^\circ}(x,y) = \frac{\partial}{\partial x} e^{-(x^2+y^2)} = -2xe^{-(x^2+y^2)}$$

(Note: Constants are often omitted in basis formulations to focus on the geometric shape of the filter). Here, the $-2x$ term forces the filter to have positive weights on one side of the vertical axis and negative weights on the other, making it perfectly designed to detect vertical transitions.

To find edges at any arbitrary angle θ, we linearly combine the x and y basis filters:

$$G_1^{\theta}(x,y) = \cos(\theta) G_1^{0^\circ} + \sin(\theta) G_1^{90^\circ}$$

This means we only need to compute the x and y derivatives once, and we can find edges at any angle by mixing them with sine and cosine weights.

5 Local Features: Corners and Autocorrelation

5.1 Informal Explanation

While an edge is a line, a corner is an intersection. If you look at a flat wall through a tiny window and move the window slightly, the view doesn't change. If you look at a straight edge and move parallel to it, the view doesn't change. But if you put that window over a *corner* and move it in *any* direction, the visual content shifts drastically. Autocorrelation is the mathematical way of shifting this window and measuring how severely the image changes.

5.2 Formal Mathematical Formulation

We measure the stability of an image patch I by calculating the Summed Square Difference (SSD) error when the patch is shifted by a small distance (u, v).

$$E_{AC}(u,v) = \sum_{x_i, y_i} [I(x_i + u, y_i + v) - I(x_i, y_i)]^2$$

To make this computationally feasible for small shifts, we use a First-Order Taylor Series Expansion to approximate the shifted image:

$$I(x_i + u, y_i + v) \approx I(x_i, y_i) + \nabla I(x_i, y_i) \cdot \begin{bmatrix} u \\ v \end{bmatrix}$$

Substituting this back into the error function yields a quadratic approximation:

$$E_{AC}(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} A \begin{bmatrix} u \\ v \end{bmatrix}$$

Where A is the **Autocorrelation matrix** (or Structure Tensor):

$$A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

Breaking down the terms:

- I_x, I_y: The image gradients in the x and y directions.
- $\sum I_x^2$: The sum of the squared horizontal gradients in the window.
- $\sum I_y^2$: The sum of the squared vertical gradients in the window.
- $\sum I_x I_y$: The cross-product term, which captures diagonal structures.

Eigenvalue Analysis: We analyze the matrix A by finding its eigenvalues, λ_1 and λ_2 (where $\lambda_2 > \lambda_1$). These eigenvalues represent the principal curvatures of the error surface—essentially telling us how fast the error grows when we shift the window.

- **Corner:** Both λ_1 and λ_2 are large. Shifting the window in *any* direction causes a massive error.
- **Edge:** $\lambda_2 \gg \lambda_1$. Shifting perpendicular to the edge causes a large error, but shifting parallel to the edge causes almost no error.
- **Flat region:** Both λ_1 and λ_2 are close to zero. Shifting the window in any direction causes almost no change.

6 Image Segmentation by Thresholding

6.1 Definition and Choice of Global Threshold

Informal Explanation: Segmentation divides an image into meaningful regions (like separating an object from its background). Thresholding is the simplest approach: "Any pixel darker than a specific shade of gray is one region; anything lighter is the other." The primary challenge is choosing that magic threshold number.

As clearly illustrated in your provided Brightness Histogram, a reliable way to find this number is by looking for a "valley" between two distinct peaks (a bimodal distribution). The sharp peak on the left (around values 10-20) represents a massive cluster of dark pixels, while the broader peak on the right (around 150-180) represents the brighter pixels. The dashed vertical line drawn at approximately 90 perfectly demonstrates the ideal choice for a global threshold: it sits right in the valley, creating a boundary that cleanly separates the dark elements from the bright elements.

Formal Mathematical Formulation: Thresholding converts a grayscale image $f(x, y)$ into a binary image $g(x, y)$ using a global threshold value T. Based on the provided histogram, this would be set as $T \approx 90$.

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases}$$

The probability of occurrence of intensity level i in an image is given by the normalized histogram (which represents the exact blue curve shown in your image):

$$p_i = \frac{n_i}{N}$$

where n_i is the number of pixels with intensity i (the height on the y-axis), and N is the total number of pixels. A global threshold T is chosen by analyzing this probability distribution p_i, explicitly searching for the local minimum between the two prominent modes, just as the dashed line indicates.

6.2 Global Automatic Thresholding: Otsu's Method

Informal Explanation: Guessing the threshold T manually by looking at a histogram is unreliable. Otsu's method is an algorithm that finds the *perfect* threshold automatically. It tests every possible shade of gray (0 to 255) and calculates a score. It finds the point where the foreground and background pixel groups are as distinctly separated from each other as possible (maximizing "between-class variance").

Formal Mathematical Formulation (Exhaustive): Otsu's method assumes the image contains two classes of pixels: C_0 (background, intensities $[0, k]$) and C_1 (foreground, intensities $[k + 1, L - 1]$).

For any chosen threshold k, we define the probabilities of class occurrence:

$$\omega_0(k) = \sum_{i=0}^{k} p_i \quad \text{and} \quad \omega_1(k) = \sum_{i=k+1}^{L-1} p_i = 1 - \omega_0(k)$$

The mean intensity values for each class are:

$$\mu_0(k) = \sum_{i=0}^{k} i \frac{p_i}{\omega_0(k)} \quad \text{and} \quad \mu_1(k) = \sum_{i=k+1}^{L-1} i \frac{p_i}{\omega_1(k)}$$

The total mean intensity of the entire image is:

$$\mu_T = \sum_{i=0}^{L-1} i p_i = \omega_0(k)\mu_0(k) + \omega_1(k)\mu_1(k)$$

The goal of Otsu's method is to maximize the *between-class variance* (σ_B^2). The variance between the two classes is defined as the squared distance of the class means from the total mean, weighted by their probabilities:

$$\sigma_B^2(k) = \omega_0(k)(\mu_0(k) - \mu_T)^2 + \omega_1(k)(\mu_1(k) - \mu_T)^2$$

Through algebraic simplification, this can be written in a much faster-to-compute form:

$$\sigma_B^2(k) = \omega_0(k)\omega_1(k)(\mu_1(k) - \mu_0(k))^2$$

The Algorithm: Otsu's method calculates $\sigma_B^2(k)$ for every possible threshold k from 0 to $L-1$. The optimal threshold T is the value of k that maximizes $\sigma_B^2(k)$. (Note: Maximizing between-class variance is mathematically equivalent to minimizing the within-class variance).

7 Image Segmentation by Clustering: K-Means

7.1 Informal Explanation

If an image has multiple colored objects (e.g., a green frog on a brown log against a blue sky), binary thresholding fails. K-Means clustering is like picking K random colors, assigning every pixel to whichever of those K colors it is closest to, and then averaging the colors in each group to find a better center. You repeat this until the groups stop changing.

7.2 Formal Mathematical Formulation

K-means partitions N observations (pixels) into K clusters ($S = \{S_1, S_2, \ldots, S_K\}$). The algorithm minimizes the *Sum of Squared Errors (SSE)* objective function:

$$J = \sum_{j=1}^{K} \sum_{x_i \in S_j} ||x_i - c_j||^2$$

Breaking down the terms:

- J: The total sum of squared distances (the error we want to minimize).
- K: The predetermined number of clusters.
- x_i: The feature vector of the i-th pixel (e.g., RGB values).
- c_j: The centroid (mean) of cluster j.
- $||x_i - c_j||^2$: The squared Euclidean distance between a pixel's features and its assigned cluster centroid.

7.3 Pros, Cons, and Choosing K

Pros:

- Conceptually simple and relatively fast to compute.
- Guarantees convergence to a local minimum.
- Easily adapts to new feature spaces (like adding spatial x, y coordinates to RGB).

Cons:

- **Requires K in advance:** The algorithm cannot figure out how many objects are in the image on its own.
- **Sensitive to Initialization:** Randomly picking bad starting centroids can cause the algorithm to get stuck in a bad local minimum.
- **Assumes Spherical Clusters:** It uses Euclidean distance, meaning it struggles if the true data clusters are elongated, irregular, or overlapping.
- **Sensitive to Outliers:** Extreme pixel values can drag the mean (c_j) heavily.

How to Choose K:

1. **Domain Knowledge:** Often in image processing, we know what we are looking for (e.g., $K = 2$ for foreground/background, $K = 3$ for sky/grass/road).
2. **The Elbow Method:** Run K-means for multiple values of K (e.g., $K = 1$ to 10). Plot the final SSE (J) against K. The error will always decrease as K increases. The optimal K is typically at the "elbow" of the curve, where adding more clusters no longer significantly reduces the error.

8 Superpixel Computation: SLIC

8.1 Informal Explanation

Superpixels are small, irregularly shaped "honeycomb" blocks of pixels that share similar colors and are physically close. SLIC (Simple Linear Iterative Clustering) is a highly efficient way to group pixels into these superpixels. It acts like a localized version of K-means, meaning it groups pixels quickly without comparing every pixel to the other side of the image.

8.2 Formal Mathematical Formulation

SLIC adapts K-means by operating in a 5-dimensional feature space: $[l, a, b, x, y]$, where l, a, b are from the CIELAB color space, and x, y are spatial coordinates.

Because color space and physical space have different scales, SLIC introduces a unified distance measure D:

$$D = \sqrt{d_c^2 + \left(\frac{m}{S} d_s\right)^2}$$

Where the color distance d_c and spatial distance d_s are:

$$d_c = \sqrt{(l_j - l_i)^2 + (a_j - a_i)^2 + (b_j - b_i)^2}$$

$$d_s = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

Breaking down the terms:

- D: The final distance score used to assign pixels to superpixels.
- S: The grid interval, roughly the expected width of a superpixel ($S = \sqrt{N/K}$).
- m: A compactness parameter. If m is large, spatial distance dominates, resulting in highly compact, grid-like superpixels. If m is small, color dominates, allowing superpixels to adhere strictly to image boundaries.